Algoritmo Dígito por Dígito para Cálculo de Raiz Quadrada no ATmega328

Gabriel Luduvice Nunes Wolfovitch, João Pedro Nascimento Monteiro,
João Vítor de Araújo Costa de Oliveira e Lucas Miranda Campos
Departamento de Engenharia Elétrica e Computação (DEEC)
Universidade Federal da Bahia (UFBA)
Salvador, Brasil
{email1, email2, email3, email4}@ufba.br

***Resumo*—O cálculo de raízes quadradas em sistemas digitais e FPGAs frequentemente esbarra no alto custo computacional e no consumo excessivo de portas lógicas quando implementado através de algoritmos tradicionais que exigem blocos de divisão e multiplicação. Este artigo apresenta o desenvolvimento, a justificativa e a validação de um algoritmo de extração de raiz quadrada binária baseado no método de deslocamento e subtração (shift-and-subtract). A metodologia adotou um fluxo de projeto estruturado: iniciou-se com a modelagem matemática em linguagem C para comprovação da estabilidade do algoritmo, seguida por uma transcrição em nível de arquitetura utilizando a linguagem Assembly. O protótipo em Assembly atuou como uma prova de conceito fundamental, mapeando o comportamento exato dos registradores e das flags de controle. Os resultados comprovam que a eliminação de operadores aritméticos complexos torna a transição para a linguagem de descrição de hardware (Verilog) direta, previsível e altamente otimizada.**

***Index Terms*—Raiz Quadrada, Assembly, Linguagem C, FPGA, Verilog, Otimização de Hardware, Sistemas Digitais.**

I. Introdução

O desenvolvimento de sistemas embarcados e o processamento de sinais em FPGAs (*Field-Programmable Gate Arrays*) exigem algoritmos aritméticos altamente otimizados. Em linguagens de alto nível, calcular uma raiz quadrada é uma tarefa trivial, resolvida por chamadas de bibliotecas padrão (como a `math.h` em C). Contudo, ao transpor essa operação para o nível de hardware, o uso de divisores e multiplicadores dedicados consome fatias significativas de DSP (*Digital Signal Processing*) e aumenta o atraso combinacional do circuito.

Para mitigar esse problema, este trabalho propõe a implementação de um algoritmo ”bit a bit”focado estritamente em operações elementares. O objetivo principal é demonstrar que a tradução de um modelo funcional em linguagem C para um protótipo operacional em Assembly garante a confiabilidade do caminho de dados (*datapath*), preparando o sistema para uma futura síntese em hardware livre de gargalos lógicos.

II. Justificativa do Método Bit a Bit

A escolha do algoritmo de extração de raiz quadrada binária dígito a dígito (também conhecido como *shift-and-subtract*) fundamenta-se na economia de recursos de hardware. Este método assemelha-se ao algoritmo manual de extração de raízes, adaptado para a base 2.

A grande vantagem arquitetural dessa escolha é que ela abole completamente o uso de divisões e multiplicações. Toda a lógica matemática é resolvida através de:

- **Deslocamentos lógicos (*Shifts*):** Operações que, em hardware, custam zero ciclos de clock adicionais, consistindo apenas em roteamento de fios no Verilog.
- **Subtrações simples:** Implementadas facilmente através de somadores em complemento de 2.
- **Testes de condição:** Avaliação direta do bit de sinal para tomadas de decisão.

Dessa forma, o custo computacional passa a ser linear em relação ao número de bits do radicando, tornando o tempo de execução previsível e compatível com máquinas de estado finito (FSM).

III. Modelagem em Alto Nível: Linguagem C

A primeira etapa do projeto consistiu em validar a matemática do método *shift-and-subtract* isoladamente da arquitetura. Para isso, o algoritmo foi implementado na linguagem C. A Listagem 1 demonstra a lógica principal (adaptada para um radicando de 8 bits).

Listing 1. Implementação funcional em C do algoritmo bit a bit.

```c
#include <stdio.h>
#include <stdint.h>

uint8_t isqrt_8bit(uint8_t X) {
    uint8_t Q = 0; // Raiz
    uint8_t A = 0; // Acumulador/Resto parcial
    int16_t T = 0; // Teste de sinal

    // Loop de 4 interacoes para radicando de 8 bits
    for(int i = 3; i >= 0; i--) {
        // Desloca A e insere os 2 bits mais
            significativos de X
        A = (A << 2) | ((X >> (i * 2)) & 0x03);

        // Define T = A - (Q concatenado com 01)
        T = A - ((Q << 2) | 1);

        Q = (Q << 1); // Desloca Q previamente

        // Testa se a subtracao foi positiva ou zero
        if(T >= 0) {
            A = T;     // Atualiza o acumulador
            Q = Q | 1; // Seta o bit menos
                significativo de Q
        }
```

```
    }
    return Q;
}
```

A execução deste código em C comprovou a eficácia do método. A lógica iterativa calcula pares de bits sequencialmente, reduzindo progressivamente o radicando original (X) enquanto constrói o resultado (Q). No entanto, variáveis em C não refletem os limites de registradores físicos e *flags* de status de uma ALU (*Arithmetic Logic Unit*), tornando a etapa seguinte mandatória.

IV. Prova de Conceito em Baixo Nível: O Protótipo em Assembly

Com a matemática validada, o algoritmo foi transcrito para a linguagem Assembly no arquivo `main.asm`. Esta etapa serviu como um protótipo em escala reduzida e foi crucial para o projeto, atuando como o verdadeiro elo de ligação entre o software e o hardware.

No `main.asm`, as abstrações da linguagem C foram substituídas pelo rigor dos componentes de hardware. Definiram-se quatro registradores fundamentais: 1) **X:** O radicando de entrada. 2) **A:** Registrador acumulador que guarda o valor parcial em trabalho. 3) **T:** Registrador utilizado para armazenar o resultado do teste de subtração. 4) **Q:** Registrador responsável por armazenar a raiz quadrada final.

No Assembly, a operação de deslocamento bit a bit (`A = (A << 2) ...`) foi traduzida para instruções nativas de *Logical Shift Left* (LSL). O grande diferencial do teste em Assembly ocorreu na avaliação da condição `if(T >= 0)`. Em vez de uma avaliação lógica de alto nível, a verificação passou a depender do comportamento natural da Unidade Lógica e Aritmética. Ao realizar a instrução de subtração para calcular T, a CPU atualizava suas *Flags* de status. Bastou utilizar uma instrução de desvio condicional (*Branch*) baseada no bit de sinal (ou *Carry*) para determinar se o bit correspondente de Q seria 1 ou 0.

V. Resultados da Validação

Para validar o protótipo em Assembly, realizou-se um teste de rastreamento (teste de mesa) com a entrada decimal 121 (em binário: 01111001). Como esperado para um número de 8 bits, o algoritmo resolveu o cálculo em exatamente quatro passos.

A Tabela I apresenta o estado interno dos registradores a cada iteração executada pelo código Assembly.

Tabela I
Validação dos Registradores em Assembly (Raiz de 121)

Passo	**A** (Binário)	**X** (Binário)	**T** (Binário)	**Q** (Binário)
Inicial	00000000	01111001	00000000	0000
1	00000001	11100100	00000000	0001
2	00000011	10010000	11111100	0010
3	00001110	01000000	00000101	0101
4	00010101	00000000	00000000	1011
Saída Final: Q = 1011 (Decimal 11) — Resto (A) = 0				

A execução em Assembly demonstrou que as atualizações dos bits ocorrem de forma precisa. No passo 2, a subtração gerou um T negativo (11111100 em complemento de 2), resultando num salto para a próxima etapa sem alterar A e mantendo o novo bit de Q em 0. Ao final do passo 4, o registrador Q armazenou o valor 1011 (11 em decimal), confirmando a extração exata da raiz quadrada de 121.

VI. Conclusão

Este trabalho demonstrou o processo metodológico completo de implementação de um algoritmo otimizado para hardware. A escolha pelo método *shift-and-subtract* provou-se altamente vantajosa, uma vez que prescinde de unidades divisoras e multiplicadoras.

O código em linguagem C garantiu a robustez funcional do algoritmo, enquanto a transcrição bem-sucedida para o protótipo em `main.asm` confirmou que o mapeamento de registradores e o controle de fluxo por *flags* de status são perfeitamente viáveis. Com a prova de conceito em Assembly totalmente funcional, conclui-se que o *datapath* e a máquina de estados (FSM) estão maduros e estruturalmente prontos para a etapa final: a descrição em hardware utilizando Verilog e a respectiva síntese na FPGA.

Referências

[1] D. Patterson e J. Hennessy, *Organização e Projeto de Computadores: A Interface Hardware/Software*, 5ª ed. Rio de Janeiro: Elsevier, 2017.

[2] M. M. Mano e M. D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL*, 5ª ed. Pearson, 2012.